

✓ ITEC623 – Deep Learning and Emerging Topics in Artificial Intelligence

Lab 2: Multi-Input Neural Networks and Functional API

Student Name: Emmanuel Orunta

Student ID: S00403657

Lab Overview

This lab explores the construction and evaluation of a multi-input neural network using the Keras Functional API. A wide-and-deep architecture is used for house-price prediction using the California Housing dataset.

Task 1: Dataset Preparation and Feature Splitting

```
# Importing Libraries

import os
import random
import shutil
import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

print("TensorFlow version:", tf.__version__)
print("NumPy version:", np.__version__)
```

```
TensorFlow version: 2.20.0
NumPy version: 2.1.3
```

```
# Set random seeds for reproducibility
SEED = 42

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Random seed:", SEED)
```

```
Random seed: 42
```

```
# Loading the California Housing dataset

housing = fetch_california_housing()

X = housing.data
y = housing.target

print("Dataset shape:", X.shape)
print("Target shape:", y.shape)

print("\nFeature names:")
for i, name in enumerate(housing.feature_names):
    print(f"Feature {i}: {name}")

print("\nFirst five target values:")
print(y[:5])
```

```
Dataset shape: (20640, 8)
Target shape: (20640,)
```

```
Feature names:
Feature 0: MedInc
Feature 1: HouseAge
Feature 2: AveRooms
Feature 3: AveBedrms
Feature 4: Population
Feature 5: AveOccup
Feature 6: Latitude
Feature 7: Longitude
```

```
First five target values:
[4.526 3.585 3.521 3.413 3.422]
```

```
# Splitting the dataset into training, validation and test sets
```

```
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=SEED
)
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full,
    y_train_full,
    test_size=0.20,
    random_state=SEED
)
```

```
print("Training set shape:", X_train.shape)
print("Validation set shape:", X_valid.shape)
print("Test set shape:", X_test.shape)
```

```
print("\nTraining targets:", y_train.shape)
print("Validation targets:", y_valid.shape)
print("Test targets:", y_test.shape)
```

```
Training set shape: (13209, 8)
Validation set shape: (3303, 8)
Test set shape: (4128, 8)
```

```
Training targets: (13209,)
Validation targets: (3303,)
Test targets: (4128,)
```

```
# Standardise the input features
```

```
scaler = StandardScaler()
```

```
# Fit only on the training data
X_train_scaled = scaler.fit_transform(X_train)
```

```
# Apply the same transformation to validation and test data
X_valid_scaled = scaler.transform(X_valid)
X_test_scaled = scaler.transform(X_test)
```

```
print("Training data after scaling:")
print("Mean:", np.mean(X_train_scaled, axis=0))
print("Standard deviation:", np.std(X_train_scaled, axis=0))
```

```
Training data after scaling:
Mean: [-6.48211517e-15 -4.99932361e-17  1.44758911e-15 -2.44622670e-15
 2.37106454e-17  1.89784185e-16  4.87777734e-14 -4.97414132e-13]
Standard deviation: [1.  1.  1.  1.  1.  1.  1.  1.]
```

```
# Creating the two input pathways
```

```
# Input A - Wide Path
# Features 0 to 4
X_train_A = X_train_scaled[:, 0:5]
X_valid_A = X_valid_scaled[:, 0:5]
X_test_A = X_test_scaled[:, 0:5]
```

```
# Input B - Deep Path
# Features 2 to 7
X_train_B = X_train_scaled[:, 2:8]
X_valid_B = X_valid_scaled[:, 2:8]
X_test_B = X_test_scaled[:, 2:8]

print("INPUT A - Wide Path")
print("Training:", X_train_A.shape)
print("Validation:", X_valid_A.shape)
print("Test:", X_test_A.shape)

print("\nINPUT B - Deep Path")
print("Training:", X_train_B.shape)
print("Validation:", X_valid_B.shape)
print("Test:", X_test_B.shape)
```

```
INPUT A - Wide Path
Training: (13209, 5)
Validation: (3303, 5)
Test: (4128, 5)
```

```
INPUT B - Deep Path
Training: (13209, 6)
Validation: (3303, 6)
Test: (4128, 6)
```

The wide pathway uses features 0–4, while the deep pathway uses features 2–7. The overlapping features allow information from features 2–4 to be available to both branches of the model.

The California Housing dataset contains eight numerical features describing housing and location characteristics. I split the data into training, validation, and test sets and standardised the features using the training data. Input A uses features 0–4 as the wide pathway, while Input B uses features 2–7 as the deep pathway. The overlap means features 2–4 are available to both branches. This can be useful because the wide branch can preserve a direct relationship with selected features, while the deep branch can learn more complex relationships from a different feature combination.

✓ Task 2: Multi-Input Model Architecture Design

```
# Wide input: 5 features
input_A = keras.Input(
    shape=(5,),
    name="wide_input"
)

# Deep input: 6 features
input_B = keras.Input(
    shape=(6,),
    name="deep_input"
)

print("Input A:", input_A)
print("Input B:", input_B)
```

```
Input A: <KerasTensor shape=(None, 5), dtype=float32, sparse=False, ragged=False, name=wide_input>
Input B: <KerasTensor shape=(None, 6), dtype=float32, sparse=False, ragged=False, name=deep_input>
```

```
# Wide pathway

wide_path = input_A
```

```
# Deep pathway

deep_path = layers.Dense(
    30,
    activation="relu",
    name="deep_hidden_1"
)(input_B)

deep_path = layers.Dense(
    30,
    activation="relu",
```

```
        name="deep_hidden_2"
    )(deep_path)
```

```
# Combine the wide and deep pathways

combined = layers.Concatenate(
    name="wide_deep_concat"
)([wide_path, deep_path])

# Regression output
# One neuron and no activation function
output = layers.Dense(
    1,
    name="house_price_output"
)(combined)

# Create Functional API model
model = keras.Model(
    inputs=[input_A, input_B],
    outputs=output,
    name="wide_deep_housing_model"
)

print("Model created successfully.")
```

Model created successfully.

```
# Display the model architecture
model.summary()
```

Model: "wide_deep_housing_model"

Layer (type)	Output Shape	Param #	Connected to
deep_input (InputLayer)	(None, 6)	0	-
deep_hidden_1 (Dense)	(None, 30)	210	deep_input[0][0]
wide_input (InputLayer)	(None, 5)	0	-
deep_hidden_2 (Dense)	(None, 30)	930	deep_hidden_1[0]...
wide_deep_concat (Concatenate)	(None, 35)	0	wide_input[0][0], deep_hidden_2[0]...
house_price_output (Dense)	(None, 1)	36	wide_deep_concat...

Total params: 1,176 (4.59 KB)
Total size: 1,176 (4.59 KB)

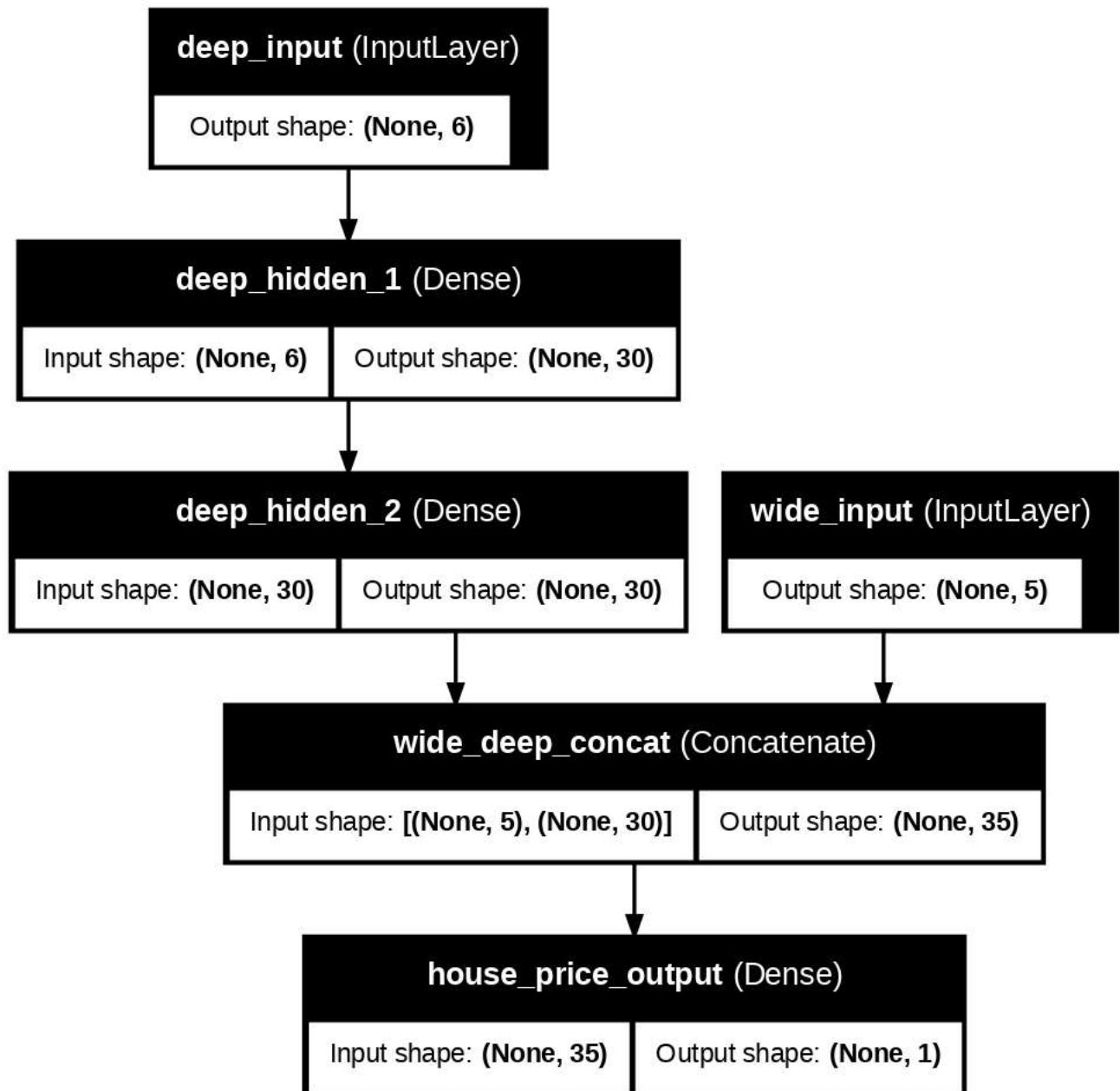
```
# Generating model architecture diagram

!pip install -q pydot graphviz

from tensorflow.keras.utils import plot_model
from IPython.display import Image, display

plot_model(
    model,
    to_file="wide_deep_architecture.png",
    show_shapes=True,
    show_layer_names=True,
    dpi=120
)

display(Image(filename="wide_deep_architecture.png"))
```



The model contains two parallel pathways. Input A is used as the wide pathway, while Input B is passed through two hidden layers before being combined with the wide pathway. The final layer produces one continuous value representing the predicted house price.

I used the Keras Functional API to create a wide-and-deep regression model with two inputs. Input A forms the wide pathway and is connected directly to the concatenation layer. Input B forms the deep pathway and passes through two Dense layers containing 30 neurons each, using ReLU activation. The outputs from the two branches are then concatenated and connected to a single linear output neuron for house-price prediction. This design follows the wide-and-deep idea because the model can learn simple relationships through the short path while the deep path learns more complex patterns.

Task 3 — Training and Evaluation

```
# Compiling the model

model.compile(
    loss="mean_squared_error",
    optimizer=keras.optimizers.SGD(
        learning_rate=1e-3
    ),
    metrics=["mae"]
)
```

```
print("Model compiled successfully.")
print("Loss: Mean Squared Error")
print("Optimizer: SGD")
print("Learning rate: 0.001")
```

```
Model compiled successfully.
Loss: Mean Squared Error
Optimizer: SGD
Learning rate: 0.001
```

```
# TensorBoard logging setup

root_logdir = "my_logs"

# Remove old logs so this experiment starts clean
if os.path.exists(root_logdir):
    shutil.rmtree(root_logdir)

os.makedirs(root_logdir, exist_ok=True)

run_id = time.strftime("%Y%m%d-%H%M%S")
run_logdir = os.path.join(root_logdir, run_id)

print("TensorBoard log directory:")
print(run_logdir)
```

```
TensorBoard log directory:
my_logs/20260827-004746
```

```
# TensorBoard callback
tensorboard_cb = keras.callbacks.TensorBoard(
    run_logdir,
    histogram_freq=1
)

print("TensorBoard callback created.")
```

```
TensorBoard callback created.
```

```
# Training the model

history = model.fit(
    [X_train_A, X_train_B],
    y_train,
    epochs=40,
    validation_data=(
        [X_valid_A, X_valid_B],
        y_valid
    ),
    callbacks=[tensorboard_cb],
    verbose=1
)
```

```
Epoch 12/40
413/413 — 3s 5ms/step - loss: 0.4738 - mae: 0.4951 - val_loss: 0.4918 - val_mae: 0.5088
Epoch 13/40
```

```

413/413 ————— 1s 4ms/step - loss: 0.4269 - mae: 0.4663 - val_loss: 0.4496 - val_mae: 0.4840
Epoch 25/40
413/413 ————— 3s 6ms/step - loss: 0.4249 - mae: 0.4652 - val_loss: 0.4478 - val_mae: 0.4830
Epoch 26/40
413/413 ————— 2s 5ms/step - loss: 0.4231 - mae: 0.4641 - val_loss: 0.4463 - val_mae: 0.4820
Epoch 27/40
413/413 ————— 2s 4ms/step - loss: 0.4214 - mae: 0.4631 - val_loss: 0.4448 - val_mae: 0.4811
Epoch 28/40
413/413 ————— 1s 4ms/step - loss: 0.4199 - mae: 0.4622 - val_loss: 0.4434 - val_mae: 0.4802
Epoch 29/40
413/413 ————— 1s 4ms/step - loss: 0.4184 - mae: 0.4614 - val_loss: 0.4422 - val_mae: 0.4795
Epoch 30/40
413/413 ————— 2s 4ms/step - loss: 0.4169 - mae: 0.4606 - val_loss: 0.4410 - val_mae: 0.4787
Epoch 31/40
413/413 ————— 2s 4ms/step - loss: 0.4156 - mae: 0.4598 - val_loss: 0.4399 - val_mae: 0.4781
Epoch 32/40
413/413 ————— 2s 4ms/step - loss: 0.4143 - mae: 0.4591 - val_loss: 0.4388 - val_mae: 0.4775
Epoch 33/40
413/413 ————— 3s 7ms/step - loss: 0.4131 - mae: 0.4584 - val_loss: 0.4377 - val_mae: 0.4769
Epoch 34/40
413/413 ————— 2s 4ms/step - loss: 0.4119 - mae: 0.4578 - val_loss: 0.4367 - val_mae: 0.4762
Epoch 35/40
413/413 ————— 2s 4ms/step - loss: 0.4108 - mae: 0.4571 - val_loss: 0.4357 - val_mae: 0.4756
Epoch 36/40
413/413 ————— 1s 4ms/step - loss: 0.4097 - mae: 0.4565 - val_loss: 0.4347 - val_mae: 0.4749
Epoch 37/40
413/413 ————— 1s 3ms/step - loss: 0.4086 - mae: 0.4559 - val_loss: 0.4338 - val_mae: 0.4744
Epoch 38/40
413/413 ————— 1s 3ms/step - loss: 0.4075 - mae: 0.4553 - val_loss: 0.4328 - val_mae: 0.4738
Epoch 39/40
413/413 ————— 2s 4ms/step - loss: 0.4065 - mae: 0.4547 - val_loss: 0.4319 - val_mae: 0.4732
Epoch 40/40
413/413 ————— 2s 5ms/step - loss: 0.4055 - mae: 0.4542 - val_loss: 0.4309 - val_mae: 0.4726

```

```
# Evaluate final performance on the test set
```

```

test_loss, test_mae = model.evaluate(
    [X_test_A, X_test_B],
    y_test,
    verbose=0
)

```

```

print("Final Test Results")
print("-----")
print(f"Test MSE: {test_loss:.6f}")
print(f"Test MAE: {test_mae:.6f}")

```

```

Final Test Results
-----
Test MSE: 0.415335
Test MAE: 0.463764

```

```
# Training and validation loss
```

```

plt.figure(figsize=(8, 5))

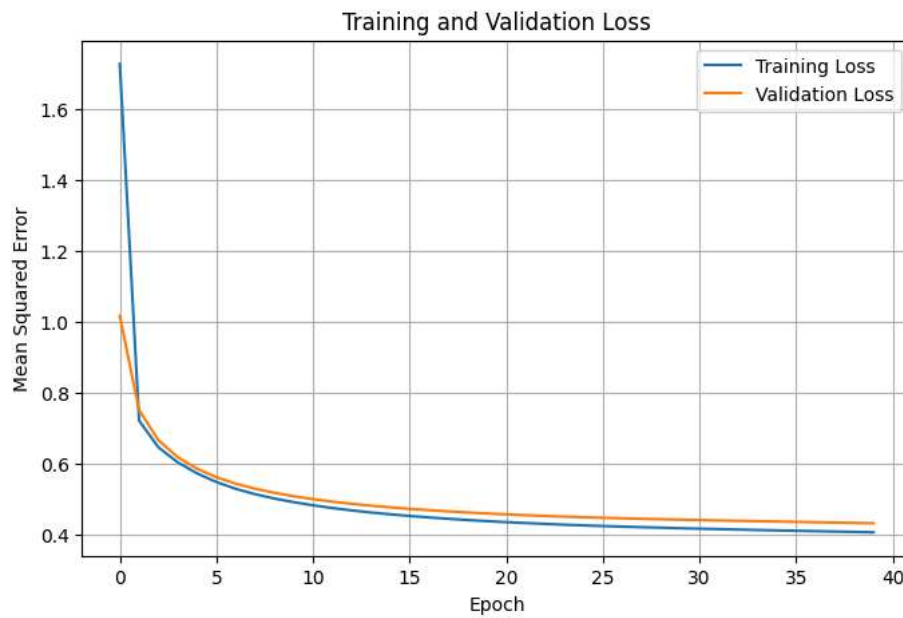
plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)

plt.show()

```



```
# Training and validation MAE

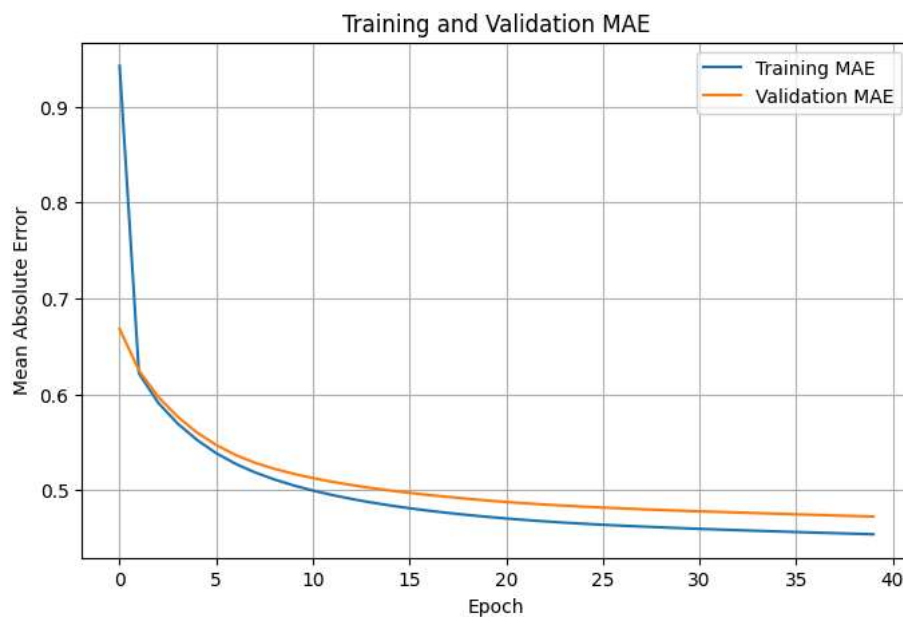
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["mae"],
    label="Training MAE"
)

plt.plot(
    history.history["val_mae"],
    label="Validation MAE"
)

plt.xlabel("Epoch")
plt.ylabel("Mean Absolute Error")
plt.title("Training and Validation MAE")
plt.legend()
plt.grid(True)

plt.show()
```



```
# Load TensorBoard
%load_ext tensorboard
```

```
# Display TensorBoard
%tensorboard --logdir my_logs
```

TensorBoard

TIME SERIES

SCALARS

GRAPHS

DISTRIBUTIONS

HISTOGRAM INACTIVE

- ☐ Show data download links
- ☐ Ignore outliers in chart scaling

Tooltip sorting method: **default**

Smoothing



0.6

Horizontal Axis

STEP

RELATIVE

WALL

Runs

Write a regex to filter runs

- ☐ 20260827-004746/train
- ☐ 20260827-004746/validation

TOGGLE ALL RUNS

my_logs

Filter tags (regular expressions supported)

epoch_learning_rate

epoch_learning_rate
tag: epoch_learning_rate

epoch_loss

epoch_loss
tag: epoch_loss

Alt + Scroll to Zoom

The model was trained for 40 epochs using stochastic gradient descent with a learning rate of 0.001 and mean squared error as the loss function. I used TensorBoard to monitor the training and validation performance during training. The loss decreased quickly during the early epochs and then gradually became more stable towards the end. The training and validation curves followed a similar pattern and remained reasonably close to each other. This suggests that the model was learning useful patterns without showing strong signs of overfitting. The final test MSE was 0.415335, while the test MAE was 0.463764.

The curves remained reasonably close, with the validation loss slightly higher than the training loss towards the end. There was no strong indication of severe overfitting.

Task 4 — Prediction and Interpretation

```
# Prediction and Interpretation

num_samples = 10

# Unseen test samples
X_new_A = X_test_A[:num_samples]
X_new_B = X_test_B[:num_samples]

# True target values
y_true = y_test[:num_samples]
```

```
print("Input A shape:", X_new_A.shape)
print("Input B shape:", X_new_B.shape)
print("True labels shape:", y_true.shape)
```

```
Input A shape: (10, 5)
Input B shape: (10, 6)
True labels shape: (10,)
```

```
# Generate predictions
y_pred = model.predict(
    [X_new_A, X_new_B],
    verbose=0
).flatten()

print("Predicted values:")
print(y_pred)

print("\nTrue values:")
print(y_true)
```

```
Predicted values:
[0.34152666 2.0896373 3.129177 2.763102 3.0269108 1.8871489
 2.7088008 1.8171124 2.890925 3.9347067 ]

True values:
[0.477 0.458 5.00001 2.186 2.78 1.587 1.982 1.575 3.4
 4.466 ]
```

```
# Compare predicted and actual values

prediction_table = pd.DataFrame({
    "Actual Value": y_true,
    "Predicted Value": y_pred,
    "Absolute Error": np.abs(y_true - y_pred)
})
```

```
prediction_table
```

	Actual Value	Predicted Value	Absolute Error	
0	0.47700	0.341527	0.135473	
1	0.45800	2.089637	1.631637	
2	5.00001	3.129177	1.870833	
3	2.18600	2.763102	0.577102	
4	2.78000	3.026911	0.246911	
5	1.58700	1.887149	0.300149	
6	1.98200	2.708801	0.726801	
7	1.57500	1.817112	0.242112	
8	3.40000	2.890925	0.509075	
9	4.46600	3.934707	0.531293	

Next steps: [Generate code with prediction_table](#) [New interactive sheet](#)

The variance in the prediction table occurs because the model cannot predict every house price perfectly and some test observations are harder for it to estimate than others. For example, observations 1 and 2 have larger errors, which could be due to feature combinations or factors affecting house prices that the model has not fully learned.

```
# Calculate prediction errors

prediction_mae = np.mean(
    np.abs(y_true - y_pred)
)

prediction_rmse = np.sqrt(
    np.mean((y_true - y_pred) ** 2)
)

print(f"Prediction subset MAE: {prediction_mae:.6f}")
print(f"Prediction subset RMSE: {prediction_rmse:.6f}")
```

```
print("Prediction subset MAE: ", prediction_mae, " ")
```

```
Prediction subset MAE: 0.677139
Prediction subset RMSE: 0.882780
```

```
# Actual vs Predicted Values

plt.figure(figsize=(7, 6))

plt.scatter(
    y_true,
    y_pred
)

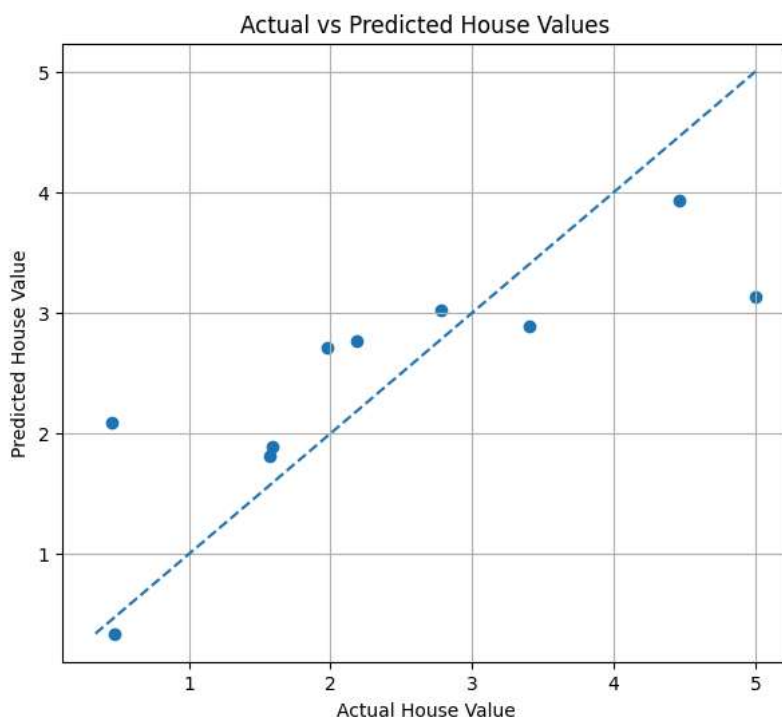
minimum = min(
    y_true.min(),
    y_pred.min()
)

maximum = max(
    y_true.max(),
    y_pred.max()
)

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    linestyle="--"
)

plt.xlabel("Actual House Value")
plt.ylabel("Predicted House Value")
plt.title("Actual vs Predicted House Values")
plt.grid(True)

plt.show()
```



I used ten observations from the test set as an unseen subset and generated house-price predictions using both input pathways. The comparison showed that some predictions were close to the true values, while others had larger errors. These differences are expected because the model only learns patterns from the available training features and cannot capture every factor that influences house prices. Errors may also be larger for observations that are less similar to examples seen during training. Overall, the predictions indicate that the model learned useful relationships, although the results are not expected to be perfect.

Task 5 — Architectural Reasoning and Extensions

The wide-and-deep architecture provides two different pathways for processing the input features. The wide branch provides a direct path, while the deep branch can learn more complex relationships through its hidden layers.

A multi-input model may outperform a single-input model when different groups of features contain useful information that can be processed in different ways. The wide branch can retain direct relationships, while the deep branch can learn more complex interactions. Similar architectures could be useful in e-commerce, healthcare, finance, recommendation systems, or any application combining different feature groups. The model could be extended by adding more input branches, additional engineered features, regularization such as dropout, or batch normalization. Other optimisers and learning rates could also be tested systematically to determine whether the architecture can achieve better validation and test performance. This separation can make the architecture easier to interpret because each branch has a clear role before the branches are combined.

Task 6 — Experimental Control and Reproducibility

```
# Reproducibility

def create_wide_deep_model():

    # Wide input
    input_A = keras.Input(
        shape=(5,),
        name="wide_input"
    )

    # Deep input
    input_B = keras.Input(
        shape=(6,),
        name="deep_input"
    )

    # Wide pathway
    wide_path = input_A

    # Deep pathway
    deep_path = layers.Dense(
        30,
        activation="relu",
        name="deep_hidden_1"
    )(input_B)

    deep_path = layers.Dense(
        30,
        activation="relu",
        name="deep_hidden_2"
    )(deep_path)

    # Concatenate branches
    combined = layers.Concatenate(
        name="wide_deep_concat"
    )([wide_path, deep_path])

    # Regression output
    output = layers.Dense(
        1,
        name="house_price_output"
    )(combined)

    # Create model
    new_model = keras.Model(
        inputs=[input_A, input_B],
        outputs=output,
        name="wide_deep_housing_model"
    )

    # Compile model
    new_model.compile(
        loss="mean_squared_error",
        optimizer=keras.optimizers.SGD(
            learning_rate=1e-3
```

```

    ),
    metrics=["mae"]
)

return new_model

```

```

# Reset all random seeds
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Python seed reset:", SEED)
print("NumPy seed reset:", SEED)
print("TensorFlow seed reset:", SEED)

```

```

Python seed reset: 42
NumPy seed reset: 42
TensorFlow seed reset: 42

```

```

# Create a completely new model
repro_model = create_wide_deep_model()

print("Fresh model created for reproducibility experiment.")

```

```

Fresh model created for reproducibility experiment.

```

```

# Retraining the fresh model

repro_history = repro_model.fit(
    [X_train_A, X_train_B],
    y_train,
    epochs=40,
    validation_data=(
        [X_valid_A, X_valid_B],
        y_valid
    ),
    verbose=1
)

```

```

Epoch 12/40
413/413 ————— 1s 3ms/step - loss: 0.4738 - mae: 0.4951 - val_loss: 0.4918 - val_mae: 0.5088
Epoch 13/40
413/413 ————— 2s 4ms/step - loss: 0.4671 - mae: 0.4910 - val_loss: 0.4857 - val_mae: 0.5055
Epoch 14/40
413/413 ————— 2s 5ms/step - loss: 0.4613 - mae: 0.4874 - val_loss: 0.4805 - val_mae: 0.5024
Epoch 15/40
413/413 ————— 1s 3ms/step - loss: 0.4561 - mae: 0.4842 - val_loss: 0.4758 - val_mae: 0.4997
Epoch 16/40
413/413 ————— 1s 3ms/step - loss: 0.4515 - mae: 0.4813 - val_loss: 0.4716 - val_mae: 0.4973
Epoch 17/40
413/413 ————— 1s 3ms/step - loss: 0.4473 - mae: 0.4787 - val_loss: 0.4678 - val_mae: 0.4951
Epoch 18/40
413/413 ————— 1s 3ms/step - loss: 0.4435 - mae: 0.4764 - val_loss: 0.4644 - val_mae: 0.4931
Epoch 19/40
413/413 ————— 1s 3ms/step - loss: 0.4400 - mae: 0.4743 - val_loss: 0.4613 - val_mae: 0.4912
Epoch 20/40
413/413 ————— 1s 3ms/step - loss: 0.4369 - mae: 0.4724 - val_loss: 0.4585 - val_mae: 0.4894

```

```

Epoch 32/40
413/413 ————— 1s 3ms/step - loss: 0.4143 - mae: 0.4591 - val_loss: 0.4388 - val_mae: 0.4775
Epoch 33/40
413/413 ————— 2s 4ms/step - loss: 0.4131 - mae: 0.4584 - val_loss: 0.4377 - val_mae: 0.4769
Epoch 34/40
413/413 ————— 2s 5ms/step - loss: 0.4119 - mae: 0.4578 - val_loss: 0.4367 - val_mae: 0.4762
Epoch 35/40
413/413 ————— 1s 3ms/step - loss: 0.4108 - mae: 0.4571 - val_loss: 0.4357 - val_mae: 0.4756
Epoch 36/40
413/413 ————— 1s 3ms/step - loss: 0.4097 - mae: 0.4565 - val_loss: 0.4347 - val_mae: 0.4749
Epoch 37/40
413/413 ————— 1s 3ms/step - loss: 0.4086 - mae: 0.4559 - val_loss: 0.4338 - val_mae: 0.4744
Epoch 38/40
413/413 ————— 1s 3ms/step - loss: 0.4075 - mae: 0.4553 - val_loss: 0.4328 - val_mae: 0.4738
Epoch 39/40
413/413 ————— 1s 3ms/step - loss: 0.4065 - mae: 0.4547 - val_loss: 0.4319 - val_mae: 0.4732
Epoch 40/40
413/413 ————— 1s 3ms/step - loss: 0.4055 - mae: 0.4542 - val_loss: 0.4309 - val_mae: 0.4726

```

```
# Evaluating the reproducibility model
```

```

repro_loss, repro_mae = repro_model.evaluate(
    [X_test_A, X_test_B],
    y_test,
    verbose=0
)

print("Original model")
print("-----")
print(f"Test MSE: {test_loss:.6f}")
print(f"Test MAE: {test_mae:.6f}")

print("\nReproducibility model")
print("-----")
print(f"Test MSE: {repro_loss:.6f}")
print(f"Test MAE: {repro_mae:.6f}")

print("\nDifference")
print("-----")
print(f"MSE difference: {abs(test_loss - repro_loss):.6f}")
print(f"MAE difference: {abs(test_mae - repro_mae):.6f}")

```

```

Original model
-----
Test MSE: 0.415335
Test MAE: 0.463764

Reproducibility model
-----
Test MSE: 0.415335
Test MAE: 0.463764

Difference
-----
MSE difference: 0.000000
MAE difference: 0.000000

```

```

# Reproducibility results table

reproducibility_results = pd.DataFrame({
    "Experiment": [
        "Original model",
        "Reproducibility run"
    ],
    "Test MSE": [
        test_loss,
        repro_loss
    ],
    "Test MAE": [
        test_mae,
        repro_mae
    ]
})

reproducibility_results

```

	Experiment	Test MSE	Test MAE	
0	Original model	0.415335	0.463764	
1	Reproducibility run	0.415335	0.463764	

Next steps:

[Generate code with reproducibility_results](#)[New interactive sheet](#)

The same model architecture was trained again after resetting the Python, NumPy and TensorFlow random seeds. The resulting test metrics were compared with the original experiment to assess how consistent the results were.

I set the random seeds for Python, NumPy and TensorFlow before training the model. I then reset the seeds, created a new model with the same architecture and trained it again under the same conditions. The results from both experiments were exactly the same, with a test MSE of 0.415335 and a test MAE of 0.463764 in both runs. This shows that the experiment was reproducible under the current Colab